



**MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Kushnirenko Ecaterina FAF-243

Report

Laboratory work n.4

of Computer Graphics

Checked by:

Olga Grosu, university assistant

DISA, FCIM, UTM

Chişinău – 2025

CONTENTS

INTRODUCTION.....	3
1.1. Purpose of the laboratory work.....	3
1.2. Problem statement.....	3
MATERIALS AND METHODS.....	4
2.1. The main functions or algorithms applied.....	4
2.2. The design process.....	5
2.3. Interaction techniques.....	6
RESULT.....	7
3.1. The program code.....	7
3.2. Outcomes of the program.....	34
CONCLUSION.....	35
REFERENCES.....	36

INTRODUCTION

1.1. Purpose of the laboratory work

The purpose of this laboratory work was to develop an interactive ecosystem simulation using Processing, demonstrating advanced object-oriented programming concepts including inheritance, polymorphism, and composition. The exercise aimed to create a complex system where multiple autonomous agents (creatures) interact within a digital environment, applying physics-based forces, oscillatory motion for locomotion, lifecycle management, and particle systems. Through this work, skills in simulation design, vector mathematics, behavioral algorithms, and interactive programming were developed.

1.2. Problem statement

The task required creating an ecosystem simulation where multiple creature objects move and interact according to specific rules. The following requirements were implemented:

1. **Forces:** Implement vector-based forces including attraction, repulsion, drag, steering behaviors, and boundary constraints.
2. **Environment elements:** Add at least two non-creature elements such as Food items, Predators, and Obstacles that creatures detect and respond to.
3. **Oscillating bodies:** Create creatures with oscillating body parts (wings, fins, tails) where oscillation speed and amplitude are tied to movement velocity.
4. **OOP with inheritance and polymorphism:** Design a Creature base class with at least two subclasses having different behaviors and appearances.
5. **Resources, health, and lifecycle:** Track health/energy for each creature, implement eating, death, reproduction, and competition rules.
6. **Particle systems:** Attach particle emitters to creatures for visual effects like trails, bubbles, and death/birth animations.

MATERIALS AND METHODS

2.1. The main functions or algorithms applied

The main functions used in the program are:

1. **setup()** – initializes the canvas, creates creature, food, predator, and obstacle objects with initial parameters.
2. **draw()** – continuously updates and renders all entities, handles lifecycle events, and respawns food.
3. **applyEnvironmentalForces()** – calculates and applies all external forces to creatures: drag, food attraction, predator avoidance, obstacle repulsion, separation, and boundary forces.
4. **seek()** – implements Reynolds' steering behavior for goal-directed movement toward targets.
5. **update()** – updates physics (velocity, position), oscillation parameters, health/metabolism, and particle systems for each creature.
6. **display()** – renders creatures with oscillating body parts at their current location using transformations.
7. **tryReproduce()** – checks reproduction conditions and creates offspring with inherited and mutated genes.
8. **eat()** – handles food consumption, health restoration, and eating particle effects.

Key programming concepts applied include:

1. Use of the PVector class to manage motion vectors (position, velocity, acceleration).
2. Abstract base class (Creature) with polymorphic methods overridden in subclasses (Butterfly, Fish).
3. Oscillation using sine functions with phase, speed, and amplitude parameters.
4. Particle systems using ArrayList to manage dynamic particle collections.
5. Genetic inheritance with mutation for size, speed, and color genes.
6. Force composition using vector addition and limiting for realistic physics.

2.2. The design process

The visual and behavioral design of the ecosystem was developed with the following considerations:

Creature Types:

1. **Butterfly:** A flying creature with four oscillating wings. Wings flap faster when moving quickly. Features colorful wing patterns with spots, oscillating antennae, and particle trail effects.
2. **Fish:** A swimming creature with an oscillating tail fin that drives locomotion. Features pectoral fins that paddle, a dorsal fin that sways, scale patterns, and bubble particle effects.

Environment Elements:

1. **Food:** Pulsing, glowing orbs in green, yellow, and pink colors. Size determines nutrition value. Respawns periodically.
2. **Predators:** Shark-like hunters with oscillating tails that chase and damage creatures.
3. **Obstacles:** Rocky barriers with textured appearance that creatures avoid using repulsion forces.

Forces System:

1. Drag force proportional to velocity squared for realistic fluid resistance.
2. Steering behaviors (seek/flee) for goal-directed and avoidance movement.
3. Inverse-square repulsion from obstacles.
4. Separation force to prevent creature overlap.
5. Soft boundary walls to keep creatures in the visible area.

Lifecycle System:

1. Health depletes over time (metabolism) and with movement (energy cost).
2. Eating food restores health based on food nutrition value.
3. Death occurs when health reaches zero, triggering particle explosion.
4. Reproduction occurs when health exceeds threshold, creating offspring with mutated genes.

2.3. Interaction techniques

The simulation implements several user interaction methods:

1. **Mouse Click:** Left-clicking spawns a cluster of 5 food items at the cursor position, allowing users to influence creature behavior and survival.
2. **Keyboard Controls:**
 - a. SPACE – Pause/Resume the simulation
 - b. F – Spawn food at cursor position
 - c. B – Spawn a new Butterfly at cursor position
 - d. S – Spawn a new Fish at cursor position
 - e. H – Toggle health bar visibility

RESULT

3.1. The program code

The complete program code is provided below. The code demonstrates object-oriented design with an abstract Creature base class and two concrete subclasses (Butterfly and Fish).

```
//GLOBAL VARIABLES
ArrayList<Creature> creatures;
ArrayList<Food> foods;
ArrayList<Obstacle> obstacles;
ArrayList<Predator> predators;
ParticleSystem globalParticles;

// Environment settings
float dragCoefficient = 0.02;
float globalTime = 0;

// UI state
boolean showForces = false;
boolean showHealth = true;
boolean paused = false;

// Spawn settings
int maxCreatures = 50;
int maxFood = 30;
int maxPredators = 3;

void setup() {
    size(1200, 800);

    // Initialize collections
    creatures = new ArrayList<Creature>();
    foods = new ArrayList<Food>();
    obstacles = new ArrayList<Obstacle>();
    predators = new ArrayList<Predator>();
    globalParticles = new ParticleSystem(new PVector(0, 0));

    // Create initial creatures - mix of butterflies and fish
    for (int i = 0; i < 15; i++) {
        if (random(1) > 0.5) {
```

```

        creatures.add(new Butterfly(random(width), random(height)));
    } else {
        creatures.add(new Fish(random(width), random(height)));
    }
}

// Create initial food
for (int i = 0; i < 20; i++) {
    foods.add(new Food(random(50, width-50), random(50, height-50)));
}

// Create obstacles
obstacles.add(new Obstacle(300, 300, 60));
obstacles.add(new Obstacle(700, 500, 80));
obstacles.add(new Obstacle(500, 200, 50));

// Create predators
for (int i = 0; i < 2; i++) {
    predators.add(new Predator(random(width), random(height)));
}
}

void draw() {
    // Draw gradient background
    drawBackground();

    if (!paused) {
        globalTime += 0.016; // Approximate dt

        // Update and draw obstacles
        for (Obstacle obs : obstacles) {
            obs.display();
        }

        // Update and draw food
        for (int i = foods.size() - 1; i >= 0; i--) {
            Food f = foods.get(i);
            f.update();
            f.display();
            if (f.isConsumed) {
                foods.remove(i);
            }
        }
    }
}

```



```

}

// Update and draw predators
for (Predator p : predators) {
    p.update(creatures);
    p.display();
}

// Update creatures with all forces
ArrayList<Creature> newCreatures = new ArrayList<Creature>();

for (int i = creatures.size() - 1; i >= 0; i--) {
    Creature c = creatures.get(i);

    // Apply environmental forces
    applyEnvironmentalForces(c);

    // Update creature
    c.update();
    c.display();

    // Check for reproduction
    Creature baby = c.tryReproduce();
    if (baby != null && creatures.size() < maxCreatures) {
        newCreatures.add(baby);
    }

    // Remove dead creatures
    if (c.isDead()) {
        // Death particles
        for (int j = 0; j < 20; j++) {
            globalParticles.addParticle(c.position.copy(), c.getColor());
        }
        creatures.remove(i);
    }
}

// Add new creatures
creatures.addAll(newCreatures);

// Update global particles
globalParticles.update();

```

```

    globalParticles.display();

    // Respawn food occasionally
    if (random(1) < 0.02 && foods.size() < maxFood) {
        foods.add(new Food(random(50, width-50), random(50, height-50)));
    }
}

// Draw UI
drawUI();
}

void drawBackground() {
    // Gradient background
    for (int y = 0; y < height; y++) {
        float inter = map(y, 0, height, 0, 1);
        color c = lerpColor(color(135, 206, 235), color(34, 139, 34, 100), inter);
        stroke(c);
        line(0, y, width, y);
    }

    // Add subtle noise texture
    noStroke();
    for (int i = 0; i < 50; i++) {
        float x = random(width);
        float y = random(height);
        fill(255, 255, 255, random(10, 30));
        ellipse(x, y, random(2, 5), random(2, 5));
    }
}

void applyEnvironmentalForces(Creature c) {
    // 1. Drag force
    PVector drag = c.velocity.copy();
    float speed = drag.mag();
    drag.normalize();
    drag.mult(-1 * dragCoefficient * speed * speed);
    c.applyForce(drag);

    // 2. Food attraction
    Food nearestFood = null;
    float nearestFoodDist = Float.MAX_VALUE;

```

```

for (Food f : foods) {
    if (!f.isConsumed) {
        float d = PVector.dist(c.position, f.position);
        if (d < nearestFoodDist && d < c.senseRadius) {
            nearestFoodDist = d;
            nearestFood = f;
        }
    }
}

if (nearestFood != null) {
    PVector steer = seek(c, nearestFood.position);
    steer.mult(c.foodAttraction);
    c.applyForce(steer);

    // Eat if close enough
    if (nearestFoodDist < c.size + nearestFood.size) {
        c.eat(nearestFood);
    }
}

// 3. Predator avoidance
for (Predator p : predators) {
    float d = PVector.dist(c.position, p.position);
    if (d < c.senseRadius * 1.5) {
        PVector flee = seek(c, p.position);
        flee.mult(-1.5); // Flee = negative seek
        c.applyForce(flee);
    }
}

// 4. Obstacle avoidance
for (Obstacle obs : obstacles) {
    float d = PVector.dist(c.position, obs.position);
    if (d < obs.radius + c.senseRadius * 0.5) {
        PVector repel = PVector.sub(c.position, obs.position);
        repel.normalize();
        repel.mult(200 / (d * d + 1)); // Inverse square repulsion
        c.applyForce(repel);
    }
}

```

```

// 5. Separation from other creatures
for (Creature other : creatures) {
    if (other != c) {
        float d = PVector.dist(c.position, other.position);
        if (d < c.size * 3) {
            PVector sep = PVector.sub(c.position, other.position);
            sep.normalize();
            sep.mult(50 / (d + 1));
            c.applyForce(sep);
        }
    }
}

// 6. Boundary forces (soft walls)
float margin = 50;
PVector boundary = new PVector(0, 0);
if (c.position.x < margin) boundary.x = margin - c.position.x;
if (c.position.x > width - margin) boundary.x = (width - margin) - c.position.x;
if (c.position.y < margin) boundary.y = margin - c.position.y;
    if (c.position.y > height - margin) boundary.y = (height - margin) -
c.position.y;
boundary.mult(0.5);
c.applyForce(boundary);
}

PVector seek(Creature c, PVector target) {
    PVector desired = PVector.sub(target, c.position);
    desired.normalize();
    desired.mult(c.maxSpeed);
    PVector steer = PVector.sub(desired, c.velocity);
    steer.limit(c.maxForce);
    return steer;
}

void drawUI() {
    // UI Panel
    fill(0, 0, 0, 150);
    noStroke();
    rect(10, 10, 250, 220, 10);

    fill(255);
    textSize(16);

```

```

textAlign(LEFT, TOP);
text("ECOSYSTEM SIMULATION", 20, 20);

textSize(12);
text("Creatures: " + creatures.size(), 20, 50);
text("Food: " + foods.size(), 20, 70);
text("Predators: " + predators.size(), 20, 90);

text("Controls:", 20, 120);
text("SPACE - Pause/Resume", 20, 140);
text("F - Spawn Food", 20, 155);
text("B - Spawn Butterfly", 20, 170);
text("S - Spawn Fish (Swimmer)", 20, 185);
text("H - Toggle Health Bars", 20, 200);

// Legend
fill(0, 0, 0, 150);
rect(width - 180, 10, 170, 130, 10);

fill(255);
textSize(14);
text("Legend:", width - 170, 20);

textSize(11);
// Butterfly indicator
fill(255, 200, 100);
ellipse(width - 160, 50, 10, 10);
fill(255);
text("Butterfly", width - 145, 45);

// Fish indicator
fill(100, 150, 255);
ellipse(width - 160, 70, 10, 10);
fill(255);
text("Fish", width - 145, 65);

// Food indicator
fill(100, 255, 100);
ellipse(width - 160, 90, 8, 8);
fill(255);
text("Food", width - 145, 85);

```

```

// Predator indicator
fill(255, 50, 50);
ellipse(width - 160, 110, 12, 12);
fill(255);
text("Predator", width - 145, 105);

// Obstacle indicator
fill(100, 100, 100);
ellipse(width - 160, 130, 12, 12);
fill(255);
text("Obstacle", width - 145, 125);

if (paused) {
  fill(255, 255, 0);
  textSize(32);
  textAlign(CENTER, CENTER);
  text("PAUSED", width/2, height/2);
}
}

void keyPressed() {
  if (key == ' ') {
    paused = !paused;
  }
  if (key == 'f' || key == 'F') {
    foods.add(new Food(mouseX, mouseY));
  }
  if (key == 'b' || key == 'B') {
    creatures.add(new Butterfly(mouseX, mouseY));
  }
  if (key == 's' || key == 'S') {
    creatures.add(new Fish(mouseX, mouseY));
  }
  if (key == 'h' || key == 'H') {
    showHealth = !showHealth;
  }
}

void mousePressed() {
  // Spawn food cluster on click
  if (mouseButton == LEFT) {
    for (int i = 0; i < 5; i++) {

```

```

        foods.add(new Food(mouseX + random(-30, 30), mouseY + random(-30, 30)));
    }
}

// ===== PARTICLE SYSTEM =====
class Particle {
    PVector position;
    PVector velocity;
    PVector acceleration;
    float lifespan;
    color col;
    float size;

    Particle(PVector pos, color c) {
        position = pos.copy();
        velocity = PVector.random2D().mult(random(1, 3));
        acceleration = new PVector(0, 0.05);
        lifespan = 255;
        col = c;
        size = random(3, 8);
    }

    void update() {
        velocity.add(acceleration);
        position.add(velocity);
        lifespan -= 4;
        size *= 0.98;
    }

    void display() {
        noStroke();
        fill(col, lifespan);
        ellipse(position.x, position.y, size, size);
    }

    boolean isDead() {
        return lifespan < 0;
    }
}

class ParticleSystem {

```

```

ArrayList<Particle> particles;
PVector origin;

ParticleSystem(PVector origin) {
    this.origin = origin.copy();
    particles = new ArrayList<Particle>();
}

void addParticle(PVector pos, color c) {
    particles.add(new Particle(pos, c));
}

void update() {
    for (int i = particles.size() - 1; i >= 0; i--) {
        Particle p = particles.get(i);
        p.update();
        if (p.isDead()) {
            particles.remove(i);
        }
    }
}

void display() {
    for (Particle p : particles) {
        p.display();
    }
}

// ===== FOOD =====
class Food {
    PVector position;
    float size;
    float nutrition;
    boolean isConsumed;
    float pulsePhase;
    color col;

    Food(float x, float y) {
        position = new PVector(x, y);
        size = random(8, 15);
        nutrition = map(size, 8, 15, 10, 30);
    }
}

```



```

isConsumed = false;
pulsePhase = random(TWO_PI);

// Different food colors
float r = random(1);
if (r < 0.33) {
    col = color(100, 255, 100); // Green
} else if (r < 0.66) {
    col = color(255, 200, 50); // Yellow/Gold
} else {
    col = color(255, 100, 150); // Pink/Berry
}
}

void update() {
    pulsePhase += 0.1;
}

void display() {
    if (!isConsumed) {
        float pulse = sin(pulsePhase) * 2;

        // Glow effect
        noStroke();
        for (int i = 3; i > 0; i--) {
            fill(col, 50);
            ellipse(position.x, position.y, size + pulse + i * 4, size + pulse + i *
4);
        }

        // Main body
        fill(col);
        ellipse(position.x, position.y, size + pulse, size + pulse);

        // Highlight
        fill(255, 200);
        ellipse(position.x - 2, position.y - 2, size * 0.3, size * 0.3);
    }
}

void consume() {
    isConsumed = true;
}

```

```

    }
}

// ===== OBSTACLE =====
class Obstacle {
    PVector position;
    float radius;
    color col;

    Obstacle(float x, float y, float r) {
        position = new PVector(x, y);
        radius = r;
        col = color(80, 70, 60);
    }

    void display() {
        // Shadow
        noStroke();
        fill(0, 50);
        ellipse(position.x + 5, position.y + 5, radius * 2, radius * 2);

        // Rock texture
        fill(col);
        ellipse(position.x, position.y, radius * 2, radius * 2);

        // Texture details
        fill(60, 50, 40);
        ellipse(position.x - radius * 0.3, position.y + radius * 0.2, radius * 0.4,
radius * 0.3);
        ellipse(position.x + radius * 0.4, position.y - radius * 0.3, radius * 0.3,
radius * 0.2);

        // Highlight
        fill(120, 110, 100);
        ellipse(position.x - radius * 0.2, position.y - radius * 0.3, radius * 0.5,
radius * 0.4);
    }
}

// ===== PREDATOR =====
class Predator {
    PVector position;

```

```

PVector velocity;
float size;
float maxSpeed;
float phase;

Predator(float x, float y) {
    position = new PVector(x, y);
    velocity = PVector.random2D().mult(1);
    size = 40;
    maxSpeed = 2.5;
    phase = random(TWO_PI);
}

void update(ArrayList<Creature> creatures) {
    phase += 0.1;

    // Hunt nearest creature
    Creature target = null;
    float minDist = Float.MAX_VALUE;

    for (Creature c : creatures) {
        float d = PVector.dist(position, c.position);
        if (d < minDist && d < 300) {
            minDist = d;
            target = c;
        }
    }

    if (target != null) {
        // Steering towards target
        PVector desired = PVector.sub(target.position, position);
        desired.normalize();
        desired.mult(maxSpeed);
        PVector steer = PVector.sub(desired, velocity);
        steer.limit(0.1);
        velocity.add(steer);

        // Eat if close
        if (minDist < size/2 + target.size/2) {
            target.health -= 50;
        }
    } else {

```

```

    // Wander
    velocity.rotate(random(-0.1, 0.1));
}

velocity.limit(maxSpeed);
position.add(velocity);

// Wrap around edges
if (position.x < -size) position.x = width + size;
if (position.x > width + size) position.x = -size;
if (position.y < -size) position.y = height + size;
if (position.y > height + size) position.y = -size;
}

void display() {
    pushMatrix();
    translate(position.x, position.y);
    rotate(velocity.heading());

    // Body
    fill(180, 30, 30);
    noStroke();

    // Main body - shark-like
    beginShape();
    vertex(size * 0.6, 0);
    vertex(-size * 0.3, -size * 0.25);
    vertex(-size * 0.5, 0);
    vertex(-size * 0.3, size * 0.25);
    endShape(CLOSE);

    // Dorsal fin
    fill(150, 20, 20);
    triangle(0, -size * 0.25, -size * 0.1, -size * 0.5, -size * 0.2, -size *
0.25);

    // Tail fin (oscillating)
    float tailAngle = sin(phase) * 0.4;
    pushMatrix();
    translate(-size * 0.5, 0);
    rotate(tailAngle);
    triangle(0, 0, -size * 0.3, -size * 0.2, -size * 0.3, size * 0.2);
}

```

```

    popMatrix();

    // Eye
    fill(255, 255, 0);
    ellipse(size * 0.3, -size * 0.05, size * 0.1, size * 0.1);
    fill(0);
    ellipse(size * 0.32, -size * 0.05, size * 0.05, size * 0.05);

    // Teeth
    fill(255);
    for (int i = 0; i < 3; i++) {
        float tx = size * 0.4 + i * 0.08;
        triangle(tx, -size * 0.05, tx + 0.05, size * 0.02, tx - 0.05, size * 0.02);
    }

    popMatrix();
}

// ===== CREATURE BASE CLASS =====
abstract class Creature {
    PVector position;
    PVector velocity;
    PVector acceleration;

    // Physical properties
    float size;
    float mass;
    float maxSpeed;
    float maxForce;

    // Behavior
    float senseRadius;
    float foodAttraction;

    // Health and lifecycle
    float health;
    float maxHealth;
    float metabolicRate; // Health lost per frame
    float reproductionThreshold;
    float reproductionCost;
    float reproductionCooldown;

```

```

float lastReproduction;

// Genetics
color baseColor;
float speedGene;
float sizeGene;

// Particle system
ParticleSystem particles;

// Oscillation
float oscillationPhase;
float oscillationSpeed;
float oscillationAmplitude;

Creature(float x, float y) {
    position = new PVector(x, y);
    velocity = PVector.random2D().mult(random(1, 2));
    acceleration = new PVector(0, 0);

    oscillationPhase = random(TWO_PI);
    oscillationSpeed = 0.1;
    oscillationAmplitude = 1.0;

    particles = new ParticleSystem(position);

    // Default genetics (overridden by subclasses)
    speedGene = random(0.8, 1.2);
    sizeGene = random(0.8, 1.2);
}

void applyForce(PVector force) {
    PVector f = PVector.div(force, mass);
    acceleration.add(f);
}

void update() {
    // Physics
    velocity.add(acceleration);
    velocity.limit(maxSpeed);
    position.add(velocity);
    acceleration.mult(0);
}

```

```

// Update oscillation based on speed
float speed = velocity.mag();
oscillationPhase += oscillationSpeed * (1 + speed * 0.5);
oscillationAmplitude = map(speed, 0, maxSpeed, 0.5, 1.5);

// Metabolism
health -= metabolicRate;
health -= speed * metabolicRate * 0.5; // Moving costs energy

// Update particle system
updateParticles();

// Age reproduction cooldown
if (lastReproduction > 0) {
    lastReproduction -= 1;
}
}

abstract void display();
abstract void updateParticles();
abstract color getColor();
abstract Creature createOffspring(float x, float y);

void eat(Food food) {
    health += food.nutrition;
    health = constrain(health, 0, maxHealth);
    food.consume();

    // Eating particles
    for (int i = 0; i < 5; i++) {
        particles.addParticle(position.copy(), food.col);
    }
}

boolean isDead() {
    return health <= 0;
}

Creature tryReproduce() {
    if (health > reproductionThreshold && lastReproduction <= 0) {
        health -= reproductionCost;
    }
}

```

```

    lastReproduction = reproductionCooldown;

    // Spawn near parent with slight offset
    float angle = random(TWO_PI);
    float dist = size * 2;
    float bx = position.x + cos(angle) * dist;
    float by = position.y + sin(angle) * dist;

    // Birth particles
    for (int i = 0; i < 15; i++) {
        particles.addParticle(new PVector(bx, by), getColor());
    }

    return createOffspring(bx, by);
}
return null;
}

void drawHealthBar() {
    if (showHealth) {
        float barWidth = size * 1.5;
        float barHeight = 4;
        float healthPercent = health / maxHealth;

        // Background
        fill(50);
        noStroke();
        rect(position.x - barWidth/2, position.y - size - 10, barWidth, barHeight);

        // Health bar with color gradient
        color healthColor = lerpColor(color(255, 0, 0), color(0, 255, 0),
healthPercent);
        fill(healthColor);
        rect(position.x - barWidth/2, position.y - size - 10, barWidth *
healthPercent, barHeight);
    }
}

// ===== BUTTERFLY SUBCLASS =====
class Butterfly extends Creature {
    float wingAngle;

```



```

color wingColor1;
color wingColor2;

Butterfly(float x, float y) {
    super(x, y);

    // Butterfly-specific properties
    size = 20 * sizeGene;
    mass = 0.5 * sizeGene;
    maxSpeed = 4 * speedGene;
    maxForce = 0.3;

    senseRadius = 150;
    foodAttraction = 1.5;

    health = 100;
    maxHealth = 100;
    metabolicRate = 0.08;
    reproductionThreshold = 80;
    reproductionCost = 40;
    reproductionCooldown = 300;
    lastReproduction = random(100, 300);

    oscillationSpeed = 0.3;

    // Colorful wings with genetic variation
    wingColor1 = color(
        200 + random(-50, 50),
        150 + random(-50, 50),
        50 + random(-30, 30)
    );
    wingColor2 = color(
        255,
        100 + random(-50, 50),
        50 + random(-30, 30)
    );
    baseColor = wingColor1;
}

// Constructor for offspring with inherited traits
Butterfly(float x, float y, Butterfly parent) {
    this(x, y);
}

```

```

// Inherit and mutate genes
this.speedGene = parent.speedGene + random(-0.1, 0.1);
this.sizeGene = parent.sizeGene + random(-0.1, 0.1);
this.speedGene = constrain(this.speedGene, 0.5, 1.5);
this.sizeGene = constrain(this.sizeGene, 0.5, 1.5);

// Apply genes
this.size = 20 * sizeGene;
this.mass = 0.5 * sizeGene;
this.maxSpeed = 4 * speedGene;

// Inherit colors with mutation
this.wingColor1 = mutateColor(parent.wingColor1);
this.wingColor2 = mutateColor(parent.wingColor2);
}

color mutateColor(color c) {
  float r = constrain(red(c) + random(-20, 20), 0, 255);
  float g = constrain(green(c) + random(-20, 20), 0, 255);
  float b = constrain(blue(c) + random(-20, 20), 0, 255);
  return color(r, g, b);
}

void display() {
  particles.display();

  pushMatrix();
  translate(position.x, position.y);

  // Rotate to face movement direction
  float angle = velocity.heading();
  rotate(angle);

  // Wing flapping - oscillation tied to locomotion!
  wingAngle = sin(oscillationPhase) * oscillationAmplitude * 0.8;

  // Body
  fill(60, 40, 20);
  noStroke();
  ellipse(0, 0, size * 0.3, size * 0.8);

```

```

// Head
fill(40, 30, 15);
ellipse(size * 0.3, 0, size * 0.25, size * 0.25);

// Antennae (oscillating)
stroke(40, 30, 15);
strokeWeight(1);
float antennaWiggle = sin(oscillationPhase * 2) * 0.2;
  line(size * 0.35, -size * 0.05, size * 0.5, -size * 0.3 + antennaWiggle *
size);
  line(size * 0.35, size * 0.05, size * 0.5, size * 0.3 - antennaWiggle * size);

// Antenna tips
noStroke();
fill(wingColor1);
ellipse(size * 0.5, -size * 0.3 + antennaWiggle * size, 4, 4);
ellipse(size * 0.5, size * 0.3 - antennaWiggle * size, 4, 4);

// Wings - upper pair
pushMatrix();
rotate(-wingAngle);
drawWing(wingColor1, wingColor2, size * 0.8, size * 0.5, -1);
popMatrix();

pushMatrix();
rotate(wingAngle);
drawWing(wingColor1, wingColor2, size * 0.8, size * 0.5, 1);
popMatrix();

// Wings - lower pair (slightly delayed oscillation)
float lowerWingAngle = sin(oscillationPhase - 0.3) * oscillationAmplitude *
0.6;

pushMatrix();
rotate(-lowerWingAngle);
drawWing(wingColor2, wingColor1, size * 0.5, size * 0.4, -1);
popMatrix();

pushMatrix();
rotate(lowerWingAngle);
drawWing(wingColor2, wingColor1, size * 0.5, size * 0.4, 1);
popMatrix();

```

```

    popMatrix();

    drawHealthBar();
}

void drawWing(color c1, color c2, float w, float h, int side) {
    noStroke();

    // Wing base
    fill(c1, 200);
    ellipse(-size * 0.1, side * h * 0.3, w, h);

    // Wing pattern
    fill(c2, 180);
    ellipse(-size * 0.15, side * h * 0.35, w * 0.5, h * 0.5);

    // Wing spots
    fill(255, 150);
    ellipse(-size * 0.05, side * h * 0.2, w * 0.15, h * 0.15);

    fill(0, 100);
    ellipse(-size * 0.2, side * h * 0.4, w * 0.1, h * 0.1);
}

void updateParticles() {
    particles.update();

    // Trail particles when moving fast
    if (velocity.mag() > maxSpeed * 0.7 && random(1) < 0.3) {
        PVector tailPos = PVector.sub(position,
PVector.mult(velocity.copy().normalize(), size * 0.5));
        particles.addParticle(tailPos, color(wingColor1, 150));
    }
}

color getColor() {
    return wingColor1;
}

Creature createOffspring(float x, float y) {
    return new Butterfly(x, y, this);
}

```

```

    }
}

// ===== FISH SUBCLASS =====
class Fish extends Creature {
    color bodyColor;
    color finColor;
    float tailPhase;

    Fish(float x, float y) {
        super(x, y);

        // Fish-specific properties
        size = 25 * sizeGene;
        mass = 1.0 * sizeGene;
        maxSpeed = 3.5 * speedGene;
        maxForce = 0.25;

        senseRadius = 120;
        foodAttraction = 1.2;

        health = 120;
        maxHealth = 120;
        metabolicRate = 0.06;
        reproductionThreshold = 100;
        reproductionCost = 50;
        reproductionCooldown = 400;
        lastReproduction = random(100, 400);

        oscillationSpeed = 0.2;
        tailPhase = random(TWO_PI);

        // Fish colors
        bodyColor = color(
            80 + random(-30, 30),
            120 + random(-30, 50),
            200 + random(-30, 55)
        );
        finColor = color(
            150 + random(-30, 30),
            180 + random(-30, 30),
            255

```

```

    );
    baseColor = bodyColor;
}

// Constructor for offspring
Fish(float x, float y, Fish parent) {
    this(x, y);

    // Inherit genes with mutation
    this.speedGene = constrain(parent.speedGene + random(-0.1, 0.1), 0.5, 1.5);
    this.sizeGene = constrain(parent.sizeGene + random(-0.1, 0.1), 0.5, 1.5);

    this.size = 25 * sizeGene;
    this.mass = 1.0 * sizeGene;
    this.maxSpeed = 3.5 * speedGene;

    // Inherit colors
    this.bodyColor = mutateColor(parent.bodyColor);
    this.finColor = mutateColor(parent.finColor);
}

color mutateColor(color c) {
    return color(
        constrain(red(c) + random(-15, 15), 0, 255),
        constrain(green(c) + random(-15, 15), 0, 255),
        constrain(blue(c) + random(-15, 15), 0, 255)
    );
}

void display() {
    particles.display();

    pushMatrix();
    translate(position.x, position.y);
    rotate(velocity.heading());

    // Tail oscillation - drives locomotion appearance
    float tailSwing = sin(oscillationPhase) * oscillationAmplitude * 0.5;

    // Tail fin
    pushMatrix();
    translate(-size * 0.4, 0);

```

```

rotate(tailSwing);
fill(finColor);
noStroke();
beginShape();
vertex(0, 0);
vertex(-size * 0.4, -size * 0.3);
vertex(-size * 0.35, 0);
vertex(-size * 0.4, size * 0.3);
endShape(CLOSE);
popMatrix();

// Body
fill(bodyColor);
noStroke();
ellipse(0, 0, size, size * 0.5);

// Scales pattern
fill(lerpColor(bodyColor, color(255), 0.1));
for (int i = 0; i < 3; i++) {
    for (int j = -1; j <= 1; j++) {
        ellipse(-size * 0.1 + i * size * 0.15, j * size * 0.12, size * 0.12, size
* 0.1);
    }
}

// Dorsal fin (oscillating slightly)
float dorsalWiggle = sin(oscillationPhase * 0.5) * 0.1;
pushMatrix();
translate(0, -size * 0.2);
rotate(dorsalWiggle);
fill(finColor);
beginShape();
vertex(-size * 0.15, 0);
vertex(0, -size * 0.25);
vertex(size * 0.1, 0);
endShape(CLOSE);
popMatrix();

// Pectoral fins (swimming motion)
float finAngle = sin(oscillationPhase + PI/4) * oscillationAmplitude * 0.3;

pushMatrix();

```

```

    translate(size * 0.1, size * 0.15);
    rotate(finAngle);
    fill(finColor, 200);
    ellipse(0, size * 0.1, size * 0.25, size * 0.1);
    popMatrix();

    pushMatrix();
    translate(size * 0.1, -size * 0.15);
    rotate(-finAngle);
    fill(finColor, 200);
    ellipse(0, -size * 0.1, size * 0.25, size * 0.1);
    popMatrix();

    // Eye
    fill(255);
    ellipse(size * 0.3, -size * 0.05, size * 0.15, size * 0.15);
    fill(0);
    ellipse(size * 0.32, -size * 0.05, size * 0.08, size * 0.08);

    // Mouth
    stroke(0, 50);
    strokeWeight(1);
    noFill();
    arc(size * 0.45, 0, size * 0.1, size * 0.1, -PI/4, PI/4);

    popMatrix();

    drawHealthBar();
}

void updateParticles() {
    particles.update();

    // Bubble particles from mouth when moving
    if (velocity.mag() > maxSpeed * 0.5 && random(1) < 0.15) {
        PVector mouthPos = PVector.add(position,
PVector.mult(velocity.copy().normalize(), size * 0.5));
        Particle bubble = new Particle(mouthPos, color(200, 220, 255, 150));
        bubble.velocity = new PVector(random(-0.5, 0.5), random(-1, -0.3));
        bubble.acceleration = new PVector(0, -0.02); // Bubbles rise
        particles.add(bubble);
    }
}

```



```
}

color getColor() {
    return bodyColor;
}

Creature createOffspring(float x, float y) {
    return new Fish(x, y, this);
}
}
```

3.2. Outcomes of the program

The program successfully simulated an interactive ecosystem with differentiated creature behaviors:

7. **Butterflies** exhibited graceful flying motion with wings that flapped faster when moving quickly. Their colorful wing patterns and trailing particles created visually appealing animations.
8. **Fish** demonstrated realistic swimming behavior with oscillating tails driving locomotion. Bubble particles rose from their mouths when swimming actively.
9. **Predators** actively hunted creatures, creating tension and survival pressure in the ecosystem.
10. **Food items** attracted creatures with steering forces, and consumption restored health with accompanying particle effects.
11. **Reproduction** created offspring with mutated genes, leading to visible variation in creature sizes, speeds, and colors over generations.
12. **Death and birth** events were accompanied by particle explosions, providing clear visual feedback.

The simulation maintained stable performance with up to 50 creatures, 30 food items, and multiple predators simultaneously. All creatures demonstrated clearly distinguishable motion behaviors based on their type, with oscillation visually appearing to drive their locomotion.

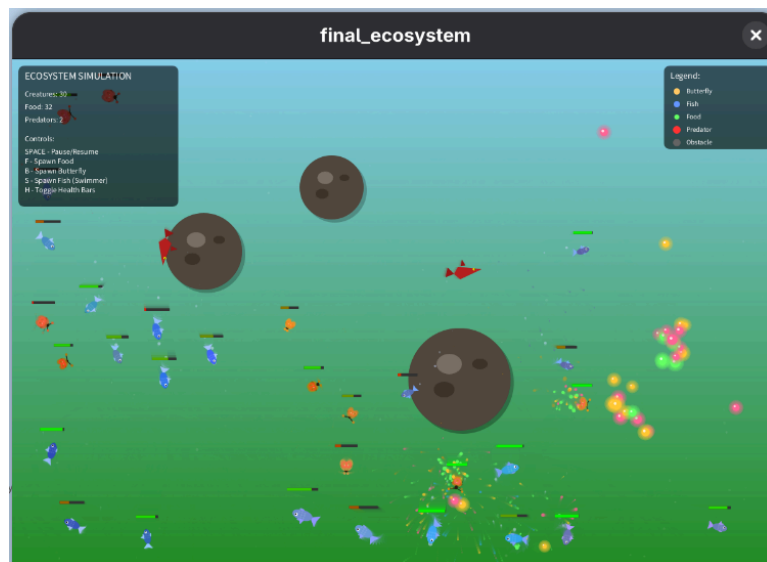


Figure 3.2.1. The ecosystem simulation showing butterflies, fish, food, predators, and obstacles

CONCLUSION

The laboratory work was completed successfully. The simulation achieved the following objectives:

1. Implementation of comprehensive PVector-based force systems including drag, steering, repulsion, and boundary constraints.
2. Creation of oscillating body parts (wings, fins, tails) with motion parameters tied to creature velocity.
3. Successful application of object-oriented programming principles with abstract base class and polymorphic subclasses.
4. Implementation of complete lifecycle management including health, metabolism, eating, death, and reproduction.
5. Integration of particle systems for visual effects (trails, bubbles, death/birth animations).
6. Implementation of genetic inheritance with mutation for emergent evolution.

Challenges encountered included balancing force magnitudes to achieve natural-looking motion, tuning metabolism rates for appropriate creature lifespans, and managing particle system performance. These were resolved through iterative parameter adjustment and optimization of particle lifetime and spawn rates.

This laboratory work demonstrated the educational value of combining physics simulation, behavioral algorithms, object-oriented design, and graphical programming. The ecosystem provides an engaging platform for observing emergent behaviors arising from simple rules, illustrating fundamental concepts in computational simulation and artificial life.

REFERENCES

1. Processing.org. "PVector Class." <https://processing.org/reference/PVector.html>
2. Shiffman, D. "The Nature of Code." <https://natureofcode.com/>
3. Processing.org. "Trigonometry Tutorial." <https://processing.org/tutorials/trig/>
4. Reynolds, C. "Steering Behaviors For Autonomous Characters." GDC 1999.
5. Processing.org. "ArrayList Reference." <https://processing.org/reference/ArrayList.html>